University of East Anglia
COMPUTING SCIENCES

Advanced Web Development

Exercise 5 Implementing Custom Credentials Authentication Systems and with Auth.js.

So far, we implemented social login using the Google Provider via Auth.js, and Postgres database using Prisma ORM.

In this lab, we are going backward to a conventional architecture: Credentials Authentication. Here, users log in using a unique identifier (like a username or email) and a secret password directly managed by your application code. While Auth.js supports this via the `CredentialsProvider`, it is explicitly designed with limitations to nudge developers toward safer defaults.

Even though username and password systems feel traditional, implementing them yourself exposes your application to a wide array of vectors that modern development frameworks actively try to avoid. For example, when handling passwords, we assume full liability for data breaches. Passwords must never be stored in plain text. We must use resource-intensive cryptographic hashing algorithms like `argon2` or `bcrypt` with unique 'salts'. To manage this sensitive credentials data, we will implement Prisma ORM as our data modelling and database access layer. You will define a User schema, run database migrations to create your tables, and use the Prisma Client to securely look up users during the login phase.

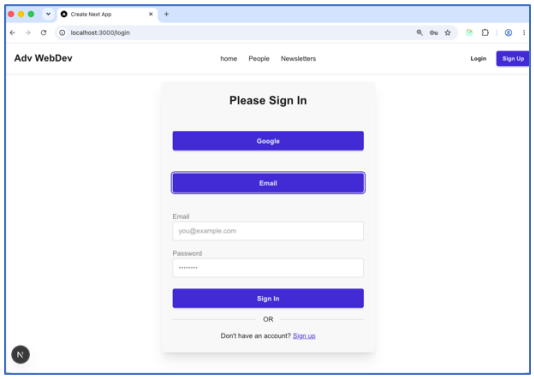
One minor configuration oversight can expose our entire database. Further, automated bots constantly scan the web, spraying leaked credential pairs at login endpoints. Protecting a custom credentials endpoint requires complex infrastructure: rate-limiting (e.g., using Redis), account lockouts, IP blacklisting, and CAPTCHAs. Google/OAuth providers handle this globally at massive scale. In addition, hand-rolled credential pipelines are susceptible to Session Hijacking, Cross-Site Scripting (XSS), and Cross-Site Request Forgery (CSRF) if cookie headers (HttpOnly, Secure, SameSite) or JWT signatures are misconfigured. Building robust 2FA/MFA (like SMS or TOTP authenticator apps) on top of a custom credentials flow requires massive backend architecture. Modern identity providers natively bundle MFA, protecting our users effortlessly.

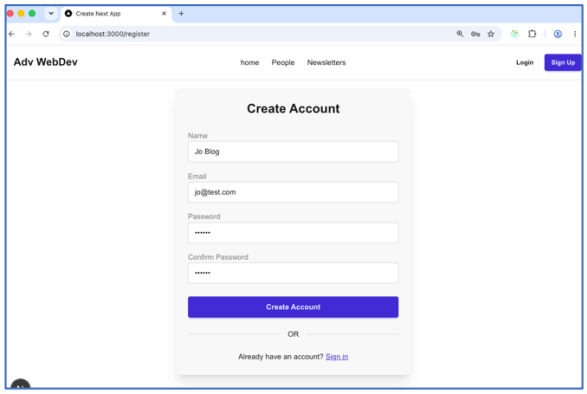
We are implementing this custom credential flow in this lab exercise to provide you with an insight into how legacy systems operate under the hood, and to understand why industry standards have shifted dramatically toward OAuth, Passkeys, and Passwordless authentication.

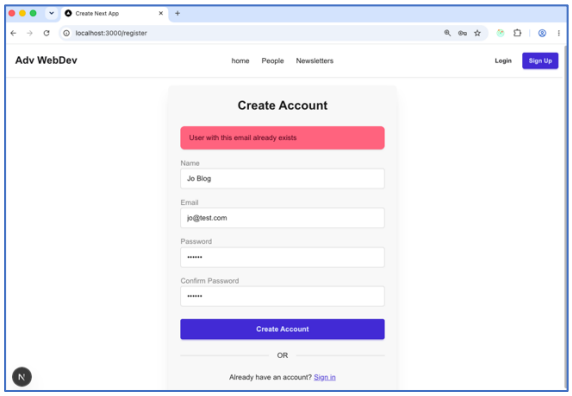
We will build upon our existing Auth.js exercises from the previous lab. Instead of relying on `GoogleProvider`, we will introduce a side-by-side configuration for a custom credential look-up via Prisma, and then integrate with Auth.js Credentials, which use JWT by default, for industry standard implementation.

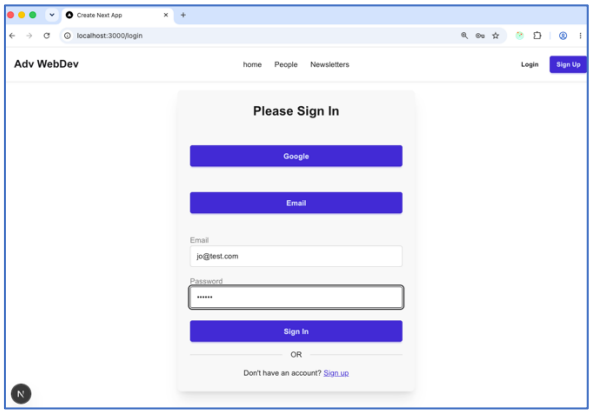
To follow this lab, you must complete Excise 4.1 " Implementing Postgres DB with Prisma ORM" first.

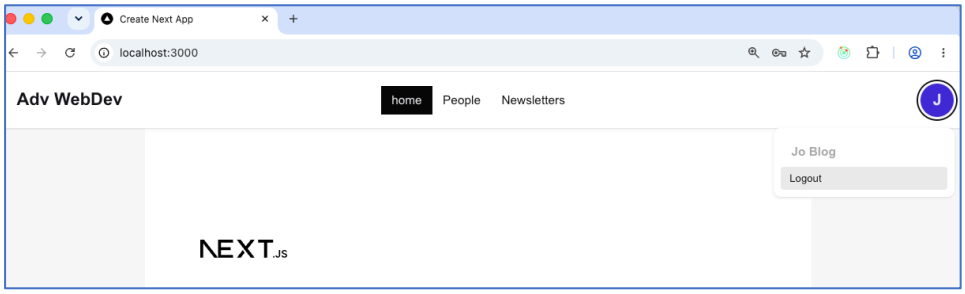
Example of intended outputs:









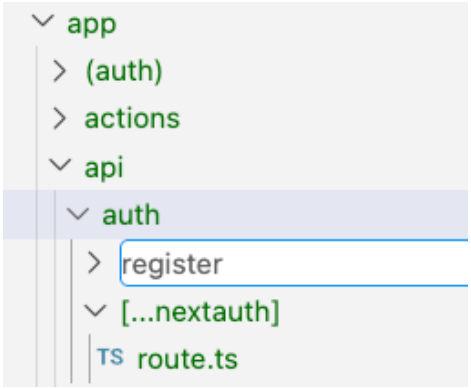


Learning Objectives:

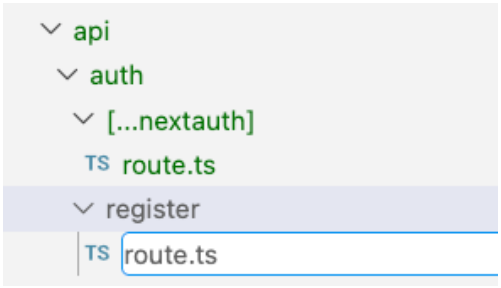
- Implement custom credentials with custom hook
- Implement web token for cookie-based authentication
- Practice RESTful API architecture design
- Implement HTTP POST request handlers
- Implement Auth.js Credentials
- Apply DaisyUI CSS

Tasks:

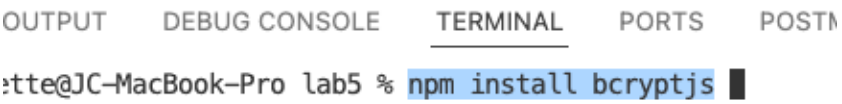
1. Create a new folder called 'lab5' and clone the code for lab4.1 inside this folder.
2. Open the folder on VSCode.
3. We will first implement Registration functionalities. Create a new folder called '`register`' inside the `app/api/auth` folder:



4. Inside this new folder, create a new file called '`route.ts`'



5. Now that we got the structure, we will first implement our custom credentials login details. First, stop the server (^C) and install `bcryptjs` by typing:



Bring up the server again.

6. Create a new file called '`auth.ts`' inside the '`lib`' folder:



7. Open this `auth.ts` file
    a. Import the following items: bcrpytjs, cookies and NextResponse:

app > lib > TS auth.ts > ...

```ts
import bcrypt from "bcryptjs";
import { cookies } from "next/headers";
import { NextResponse } from "next/server";

```

    b. Configure a salt value, I set mine to 10:

```ts

const SALT_ROUNDS = 10;
```

    c. Configure the token, I set mine to 24 hours:

```ts
const TOKEN_EXPIRY_HOURS = parseInt(
  process.env.AUTH_TOKEN_EXPIRY_HOURS || "24"
);
```

    d. Give the cookie a name:

```ts

const AUTH_COOKIE_NAME = "auth_token";
```

    e. Type the payload, here we have user id, name, email and when token expires:

```ts

export interface TokenPayload {
  userId: number;
  email: string;
  name: string;
  exp: number;
}
```

    f. Code the hash password function as follow:

```ts

/**
 * Hash a password using bcrypt
 */
export async function hashPassword(password: string): Promise<string> {
  return bcrypt.hash(password, SALT_ROUNDS);
}

```

g. Code the verifying password function as follow:

```
27  /**
28   * Verify a password against a hash
29   */
30  export async function verifyPassword(
31    password: string,
32    hash: string
33  ): Promise<boolean> {
34    return bcrypt.compare(password, hash);
35  }
36
```

h. Generate our own token, in production we should use JWT:

```
37  /**
38   * Generate a simple base64 encoded token with expiration
39   * In production, use a proper JWT library
40   */
41  export function generateToken(user: {
42    id: number;
43    email: string;
44    name: string;
45  }): string {
46    const payload: TokenPayload = {
47      userId: user.id,
48      email: user.email,
49      name: user.name,
50      exp: Date.now() + TOKEN_EXPIRY_HOURS * 60 * 60 * 1000
51    };
52
53    return Buffer.from(JSON.stringify(payload)).toString("base64");
54  }
```

i. Verifying our own token:

```
56  /**
57   * Verify and decode a token
58   */
59  export function verifyToken(token: string): TokenPayload | null {
60    try {
61      const payload = JSON.parse(
62        Buffer.from(token, "base64").toString("utf-8")
63      ) as TokenPayload;
64
65      // Check expiration
66      if (payload.exp < Date.now()) {
67        return null;
68      }
69
70      return payload;
71    } catch {
72      return null;
73    }
74  }
```

j. Set cookie :

```
76  /**
77   * Set auth cookie (httpOnly, secure in production)
78   */
79  export function setAuthCookie(response: NextResponse, token: string): void {
80    response.cookies.set(AUTH_COOKIE_NAME, token, {
81      httpOnly: true,
82      secure: process.env.NODE_ENV === "production",
83      sameSite: "lax",
84      maxAge: TOKEN_EXPIRY_HOURS * 60 * 60, // in seconds
85      path: "/"
86    });
87  }
88
```

k. We also need to clear cookie

```
89  /**
90   * Clear auth cookie
91   */
92  export function clearAuthCookie(response: NextResponse): void {
93    response.cookies.set(AUTH_COOKIE_NAME, "", {
94      httpOnly: true,
95      secure: process.env.NODE_ENV === "production",
96      sameSite: "lax",
97      maxAge: 0,
98      path: "/"
99    });
100 }
101
```

l. A function to get the token from cookie

```
102 /**
103  * Get auth token from cookies (for server components/actions)
104  */
105 export async function getAuthToken(): Promise<string | null> {
106   const cookieStore = await cookies();
107   return cookieStore.get(AUTH_COOKIE_NAME)?.value ?? null;
108 }
109
```

m. A function to get the user info from cookie

```
109
110 /**
111  * Get current user from cookie (for server components/actions)
112  */
113 export async function getCurrentUser(): Promise<TokenPayload | null> {
114   const token = await getAuthToken();
115   if (!token) return null;
116   return verifyToken(token);
117 }
```

n. A function to extract info from cookie in request

```
119  /**
120   * Extract token from cookie in request
121   */
122  export function extractTokenFromRequest(request: Request): string | null {
123    const cookieHeader = request.headers.get("cookie");
124    if (!cookieHeader) return null;
125
126    const cookies = cookieHeader.split(";").reduce(
127      (acc, cookie) => {
128        const [key, value] = cookie.trim().split("=");
129        acc[key] = value;
130        return acc;
131      },
132      {} as Record<string, string>
133    );
134
135    return cookies[AUTH_COOKIE_NAME] ?? null;
136  }
```

o. Once done, save the file.

8. Open the `route.ts` file inside the '`app/api/auth/register`' folder:
   a. Import the following items – NextResponse from next/server, prisma from lib/prisma , hashPassword, generateToken, setAuthCookie from lib/auth

```
app > api > auth > register > TS route.ts > ...
1   import { NextRequest, NextResponse } from "next/server";
2   import { prisma } from "../../../lib/prisma";
3   import { hashPassword, generateToken, setAuthCookie } from "../../../lib/auth";
4
```

   b. Code a POST route:

```
4
5   export async function POST(request: NextRequest) {
6
7   }
8
```

   c. Inside this POST route, retrieve the request body:

```
7       const body = await request.json();
8       const { email, password, name } = body;
```

d. Check if all the information is provided:

```
10    // Validate input
11    if (!email || !password || !name) {
12      return NextResponse.json(
13        { error: "Email, password, and name are required" },
14        { status: 400 }
15      );
16    }
17
```

e. Check if we know this user:

```
17
18      // Check if user already exists
19      const existingUser = await prisma.user.findUnique({
20        where: { email }
21      });
22
```

f. If the user is known to the system, reject registering them

```
22
23      if (existingUser) {
24        return NextResponse.json(
25          { error: "User with this email already exists" },
26          { status: 409 }
27        );
28      }
29
```

g. To this stage we can now create the new user. First, hash the password, and create a new entry:

```
29
30    // Hash password and create user
31    const hashedPassword = await hashPassword(password);
32    const user = await prisma.user.create({
33      data: {
34        email,
35        password: hashedPassword,
36        name
37      }
38    });
39
```

h. Generate a new token

```
39
40      // Generate token
41      const token = generateToken({
42        id: user.id,
43        email: user.email,
44        name: user.name
45      });
46
```

i. Generate the response:

```
47      // Create response with httpOnly cookie
48      const response = NextResponse.json(
49        {
50          message: "User registered successfully",
51          user: {
52            id: user.id,
53            email: user.email,
54            name: user.name
55          }
56        },
57        { status: 201 }
58      );
```

j. Finally set cookie and return the response:

```
60      // Set httpOnly cookie
61      setAuthCookie(response, token);
62
63      return response;
```

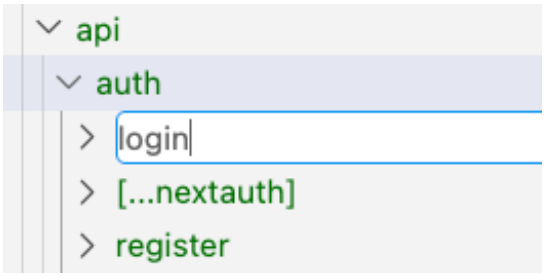
k. Place item c to item j above inside a try block

l. Handle the errors in the catch block, as follow:

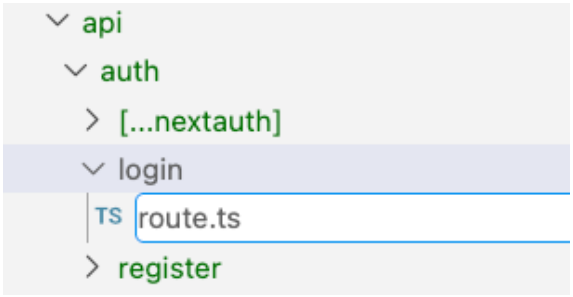
```
65    } catch (error) {
66      console.error("Registration error:", error);
67      return NextResponse.json(
68        { error: "Internal server error" },
69        { status: 500 }
70      );
71    }
```

m. Save the file.

9. For custom login, create a new folder called '`login`' inside the '`app/api/auth`' folder:



10. Inside this `login` folder, create a new file named '`route.ts`'



11. Open this `route.ts`:
    a. Import the following:

```
app > api > auth > login > TS route.ts > ...
1   import { NextRequest, NextResponse } from "next/server";
2   import { prisma } from "../../../lib/prisma";
3   import {
4     verifyPassword,
5     generateToken,
6     setAuthCookie
7   } from "../../../lib/auth";
8
```

    b. Code a POST route:

```
9   export async function POST(request: NextRequest) {
10
11  }
12
```

    c. Inside the function, retrieve the request body:

```
11
12      const body = await request.json();
13      const { email, password } = body;
14
```

d. Check if all the information is provided, email and password, if not return

```
15    // Validate input
16    if (!email || !password) {
17      return NextResponse.json(
18        { error: "Email and password are required" },
19        { status: 400 }
20      );
21    }
22
```

e. This is followed by locating the user in the system:

```
22
23    // Find user
24    const user = await prisma.user.findUnique({
25      where: { email }
26    });
27
```

f. If we can't find them, return:

```
27
28    if (!user) {
29      return NextResponse.json(
30        { error: "Invalid email or password" },
31        { status: 401 }
32      );
33    }
34
```

g. Once the user is located, verify their password, return if there's issue

```
35    // Verify password
36    const isValidPassword = await verifyPassword(password, user.password);
37    if (!isValidPassword) {
38      return NextResponse.json(
39        { error: "Invalid email or password" },
40        { status: 401 }
41      );
42    }
43
```

h. By this stage we can generate the token:

```
44    // Generate token
45    const token = generateToken({
46      id: user.id,
47      email: user.email,
48      name: user.name
49    });
50
```

i. Generate the response:

```
50
51      // Create response with httpOnly cookie
52      const response = NextResponse.json({
53        message: "Login successful",
54        user: {
55          id: user.id,
56          email: user.email,
57          name: user.name
58        }
59      });
60
```

j. Set cookie and return the response:

```
60
61      // Set httpOnly cookie
62      setAuthCookie(response, token);
63
64      return response;
65
```

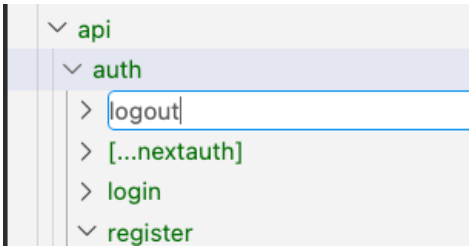
k. Again, wrap item `c` to item `j` inside a try block

l. Handle the errors in catch part:

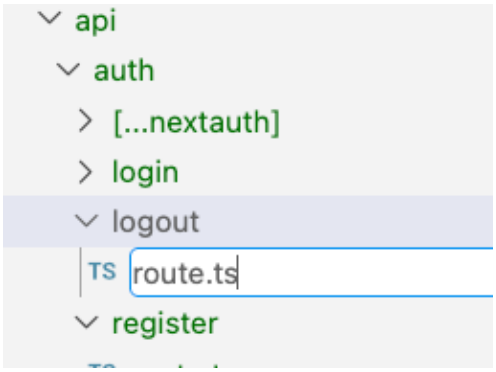
```
67    } catch (error) {
68      console.error("Login error:", error);
69      return NextResponse.json(
70        { error: "Internal server error" },
71        { status: 500 }
72      );
73    }
74
```

m. Save the file.

12. The final part is to handle custom logout. Create a new folder called '`logout`' inside the '`app/api/auth`' folder



13. Create a new file named '`route.ts`' inside this new folder:



14. Open this `route.ts` file:
    a. Import the following:

```
app > api > auth > logout > TS route.ts > ...
1   import { NextResponse } from "next/server";
2   import { clearAuthCookie } from "../../../lib/auth";
3
```

    b. Code a POST route:

```
4   export async function POST() {
5   }
6
```

    c. Inside this route, simply create a return response, clear cookie, and return.

```
4   export async function POST() {
5     const response = NextResponse.json({ message: "Logged out successfully" });
6     clearAuthCookie(response);
7     return response;
8   }
9
```

    d. Save the file.

15. The next task is to modify our custom `useAuth` hook to handle custom credentials. Open this file, and add the following:
    a. Type login credentials

```
31  interface LoginCredentials {
32    email: string;
33    password: string;
34  }
```

b. Type register credentials

```
35
36  interface RegisterCredentials extends LoginCredentials {
37    name: string;
38  }
```

c. Type the response:

```
39
40  interface AuthResponse {
41    user: User;
42    message?: string;
43    error?: string;
44  }
45
```

d. Code the login function:

```
91  const login = useCallback(
92    async (credentials: LoginCredentials): Promise<AuthResponse> => {
93      const response = await fetch("/api/auth/login", {
94        method: "POST",
95        headers: { "Content-Type": "application/json" },
96        body: JSON.stringify(credentials),
97        credentials: "include"
98      });
99
100     const data: AuthResponse = await response.json();
101
102     if (!response.ok) {
103       throw new Error(data.error || "Login failed");
104     }
105
106     // Update global state immediately
107     setAuthState({
108       user: data.user,
109       isAuthenticated: true,
110       isLoading: false
111     });
112
113     return data;
114   },
115   []
116 );
```

e. Code the register function:

```
118   const register = useCallback(
119     async (credentials: RegisterCredentials): Promise<AuthResponse> => {
120       const response = await fetch("/api/auth/register", {
121         method: "POST",
122         headers: { "Content-Type": "application/json" },
123         body: JSON.stringify(credentials),
124         credentials: "include"
125       });
126
127       const data: AuthResponse = await response.json();
128
129       if (!response.ok) {
130         throw new Error(data.error || "Registration failed");
131       }
132
133       // Update global state immediately
134       setAuthState({
135         user: data.user,
136         isAuthenticated: true,
137         isLoading: false
138       });
139
140       return data;
141     },
142     []
143   );
```

f. Add both login and register functions to the Context Type:

```
46  interface AuthContextType extends AuthState {
47    login: (credentials: LoginCredentials) => Promise<AuthResponse>;
48    register: (credentials: RegisterCredentials) => Promise<AuthResponse>;
49    checkGoogle: () => void;
50    logout: () => Promise<void>;
51  }
```

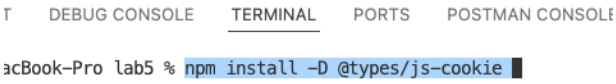
g. And add them here as well:

```
163   const value: AuthContextType = {
164     ...authState,
165     login,
166     register,
167     checkGoogle,
168     logout
169   };
170
```

h. Before we code the logout function, we need to install a package to help us grab cookie information from the client component.
    i. Stop the server (^C)
    ii. Install js-cookie



    iii. Install the type as well



    iv. Now restart the server

i. Modify the logout function, to add custom logout:
    i. First import the cookie

```
18
19    import Cookies from "js-cookie";
20
```

    ii. Modify the logout function, as follow :

```
146
147  const logout = useCallback(async () => {
148    // 1. Read cookies BEFORE calling the logout endpoint
149    const googleSessionToken =
150      Cookies.get("authjs.session-token") ||
151      Cookies.get("__Secure-authjs.session-token");
152
153    // Determine auth type before cookies are wiped
154    const isGoogleUser = Boolean(googleSessionToken);
155
156    // 2. Immediately clear client-side auth state
157    setAuthState({
158      user: user,
159      isAuthenticated: false,
160      isLoading: false
161    });
162
163    // 3. Clear server-side session/cookies
164    try {
165      await fetch("/api/auth/logout", {
166        method: "POST",
167        credentials: "include"
168      });
169    } catch {
170      // Logout anyway on error
171    }
172    // 4. Trigger Google signout conditionally without re-checking session
173    if (isGoogleUser) {
174      try {
175        await handleGoogleSignout();
176      } catch (err) {
177        console.error("Google signout error:", err);
178      }
179    }
180
181    // 4. REDIRECT HERE
182    router.push("/login");
183  }, [router]);
184
```

Note: React has an official linter rule that requires every external variable or function used inside a hook to be listed in its dependency array. Because we are calling `router.push('/login')` inside `useCallback`, the linter demands that router be included in the array.
It also prevents "Stale Closures". If Next.js updates the internal implementation of router (for instance, after a route change or layout refresh), the logout function needs to know about it.

- With [router]: If router changes, React automatically recreates the logout function so it uses the newest router instance.
- Without [router] (empty []): The logout function remains locked to the initial router instance created when the component first mounted, which can lead to broken redirects or quiet failures down the road.

16. Now that we have implemented the backend functionalities, we will need to modify the frontend form so that it works. Navigate to register form (i.e. `app/(auth)/register/page.tsx`) and open this file.
17. We will create 6 component states – name, email, password, confirm password, error and isSubmitting. We use React `useState` hook for this.
    a. Import `useState` hook

```
3   import Link from "next/link";
4   import { useState } from "react";
5
```

    b. Initialise the states (must be placed inside the component)

```
7   const [name, setName] = useState("");
8   const [email, setEmail] = useState("");
9   const [password, setPassword] = useState("");
10  const [confirmPassword, setConfirmPassword] = useState("");
11  const [error, setError] = useState("");
12  const [isSubmitting, setIsSubmitting] = useState(false);
13
```

18. next, import our custom `useAuth` hook

```
3   import Link from "next/link";
4   import { useState } from "react";
5   import { useAuth } from "../../hooks/useAuth";
6
```

19. we want 3 items from `useAuth`:

```
14  const { register, isAuthenticated, isLoading } = useAuth();
```

20. for redirect, we use `useRouter` from `next/navigation`, import it

```
3   import Link from "next/link";
4   import { useState } from "react";
5   import { useAuth } from "../../hooks/useAuth";
6   import { useRouter } from "next/navigation";
7
```

21. and use this to redirect user to homepage when login (in this lab exercise), obviously the redirect could go to user dashboard or other page. We want to check on mount, make sure `useEffect` hook is imported

```
17
18  useEffect(() => {
19    if (!isLoading && isAuthenticated) {
20      router.push("/");
21    }
22  }, [isLoading, isAuthenticated, router]);
23
```

22. code a handler function to handle the form submission:

```
18  const handleSubmit = async (e: React.SubmitEvent<HTMLFormElement>) => {
19    e.preventDefault();
20    setError("");
21
22    if (password !== confirmPassword) {
23      setError("Passwords do not match");
24      return;
25    }
26
27    if (password.length < 6) {
28      setError("Password must be at least 6 characters");
29      return;
30    }
31
32    setIsSubmitting(true);
33
34    try {
35      await register({ email, password, name });
36      router.push("/");
37    } catch (err) {
38      setError(err instanceof Error ? err.message : "Registration failed");
39    } finally {
40      setIsSubmitting(false);
41    }
42  };
```

23. attach the handler to <form> onSubmit:

```
36    return (
37      <>
38        <form onSubmit={handleSubmit} className="mt-4">
39          {" "}
```

24. check if the user is already authenticated, if so, redirect to homepage

```
46  // Redirect if already authenticated
47  if (!isLoading && isAuthenticated) {
48    router.push("/");
49    return null;
50  }
```

25. code a block to render the error message should it occur

```
54          {error && (
55            <div className="alert alert-error mb-4">
56              <span>{error}</span>
57            </div>
58          )}
```

26. add the attributes to input form control for name, set name state as well (line 69)

```
64          <input
65            type="text"
66            placeholder="Your name"
67            className="input input-bordered w-full"
68            value={name}
69            onChange={(e) => setName(e.target.value)}
70            required
71          />
```

27. same for the email:

```
78          <input
79            type="email"
80            placeholder="you@example.com"
81            className="input input-bordered w-full"
82            value={email}
83            onChange={(e) => setEmail(e.target.value)}
84            required
85          />
```

28. and same for the password:

```
92          <input
93            type="password"
94            placeholder="••••••••"
95            className="input input-bordered w-full"
96            value={password}
97            onChange={(e) => setPassword(e.target.value)}
98            required
99            minLength={6}
100         />
```

29. and same for the confirmed password:

```
106         <input
107           type="password"
108           placeholder="••••••••"
109           className="input input-bordered w-full"
110           value={confirmPassword}
111           onChange={(e) => setConfirmPassword(e.target.value)}
112           required
113         />
```

30. modify the <button> such that it is disabled when the form is submitting, and render the loader:

```
116         <button
117           type="submit"
118           className="btn btn-primary w-full"
119           disabled={isSubmitting}
120         >
121           {isSubmitting ? (
122             <span className="loading loading-spinner loading-sm"></span>
123           ) : (
124             "Create Account"
125           )}
126         </button>
```

31. Now open the custom login page (`app/(auth)/login/CustomLoginForm.tsx`):
    a. React Import `useState` hook

```
app > (auth) > login > CustomLoginForm.tsx >
1  "use client";
2  import Link from "next/link";
3  import { useState } from "react";
```

    b. Create 4 component states – `email, password, error, isSubmitting`:

```
10    const [email, setEmail] = useState("");
11    const [password, setPassword] = useState("");
12    const [error, setError] = useState("");
13    const [isSubmitting, setIsSubmitting] = useState(false);
```

    c. Import our custom hook

```
app > (auth) > login > CustomLoginForm.tsx > CustomLoginForm
1  "use client";
2  import Link from "next/link";
3  import { useState } from "react";
4  import { useAuth } from "../../hooks/useAuth";
5
```

    d. And we need 3 items from this custom hook:

```
11    const { login, isAuthenticated, isLoading } = useAuth();
12
```

    e. Utilise `useRouter` hook for redirect:

```
1  "use client";
2  import Link from "next/link";
3  import { useState } from "react";
4  import { useAuth } from "../../hooks/useAuth";
5  import { useRouter } from "next/navigation";
6
```

f. Then initialise the router:

```
13    const router = useRouter();
```

g. Same as before, check if the user has logged in, if so redirect them to homepage. We check this on mount, make sure import the `useEffect` hook.

```
14
15    useEffect(() => {
16      if (!isLoading && isAuthenticated) {
17        router.push("/");
18      }
19    }, [isLoading, isAuthenticated, router]);
20
```

h. Code a handler function to handle login form submission:

```
15    const handleSubmit = async (e: React.SubmitEvent<HTMLFormElement>) => {
16      e.preventDefault();
17      setError("");
18      setIsSubmitting(true);
19
20      try {
21        await login({ email, password });
22        router.push("/");
23      } catch (err) {
24        setError(err instanceof Error ? err.message : "Login failed");
25      } finally {
26        setIsSubmitting(false);
27      }
28    };
```

i. Attach the handler to the `<form>` `onSubmit`:

```
36    return (
37      <>
38        <form onSubmit={handleSubmit} className="mt-4">
39          {" "}
```

j. Code a block to render the error message should it occur:

```
37      <>
38        <form onSubmit={handleSubmit} className="mt-4">
39          {error && (
40            <div className="alert alert-error mb-4">
41              <span>{error}</span>
42            </div>
43          )}
44          <div className="form-control">
```

k. Add 2 attributes to form input control for email, set the email state as well (line 53)

```
48          <input
49            type="email"
50            placeholder="you@example.com"
51            className="input input-bordered w-full"
52            value={email}
53            onChange={(e) => setEmail(e.target.value)}
54            required
55          />
```

l. Same for the password:

```
61          <input
62            type="password"
63            placeholder="••••••••"
64            className="input input-bordered w-full"
65            value={password}
66            onChange={(e) => setPassword(e.target.value)}
67            required
68          />
```

m. Modify the <button> such that it is disabled when the form is submitting and present a loader:

```
71          <button
72            type="submit"
73            className="btn btn-primary w-full"
74            disabled={isSubmitting}
75          >
76            {isSubmitting ? (
77              <span className="loading loading-spinner loading-sm"></span>
78            ) : (
79              "Sign In"
80            )}
81          </button>
```

n. Save the file.

32. To get the route works properly, modify the `proxy.ts` file and add "register" path:

a. First import cookie (line 4)

```
3   import { auth } from "@/auth";
4   import { cookies } from "next/headers";
5
```

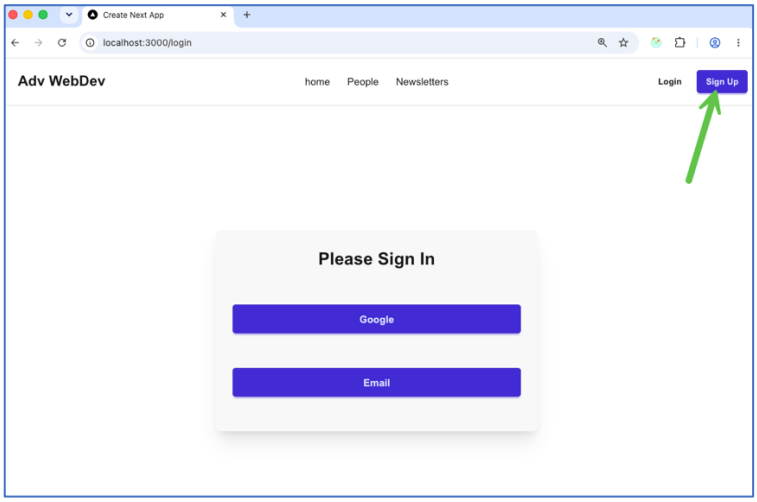
b. Make the function async so that we can use server-side cookie function, get the cookie with the custom token , then get current path. Check if user is authenticated. If they are not, and the current path is not login or register, then redirect to login page. Below is the code:

```
5
6   export const proxy = auth(async (req) => {
7     const cookieStore = await cookies();
8     const token = cookieStore.get("auth_token")?.value;
9     const currentPath = req.nextUrl.pathname;
10
11    const authenticated = req.auth !== null || token !== undefined;
12
13    if (
14      !authenticated &&
15      currentPath !== "/login" &&
16      currentPath !== "/register"
17    ) {
18      const newUrl = new URL("/login", req.nextUrl.origin);
19      return Response.redirect(newUrl);
20    }
21  });
22
```

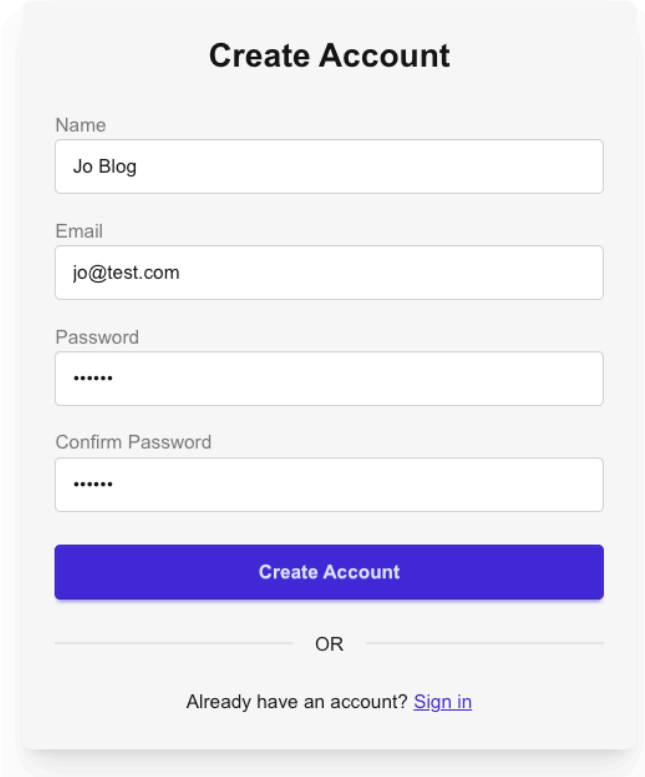
Save the file.

33. Now test it!

Step 1 - click Signup button

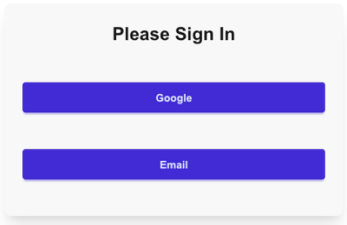


Step 2 – enter user details and click Create Account button

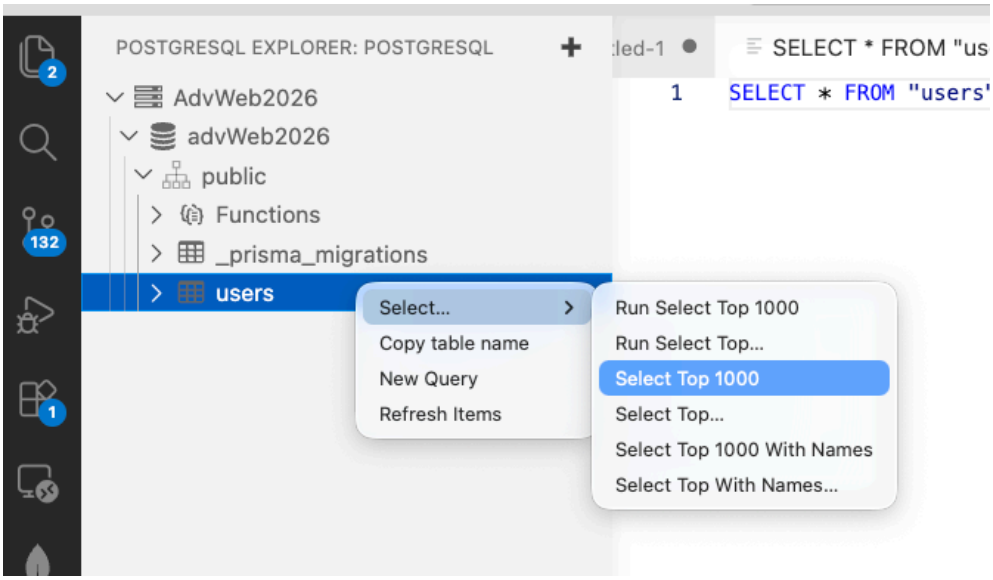


Step 3 – you will be redirect to login page

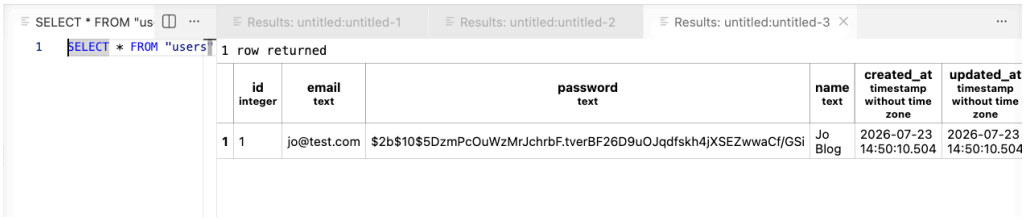




Step 4 – check the database entry:

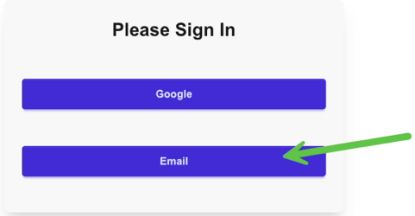


Step 5 – observe the user entry is created in the database

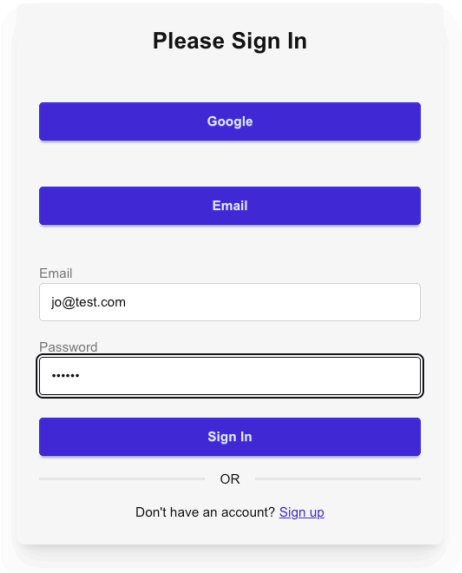


Step 6 – click Email button

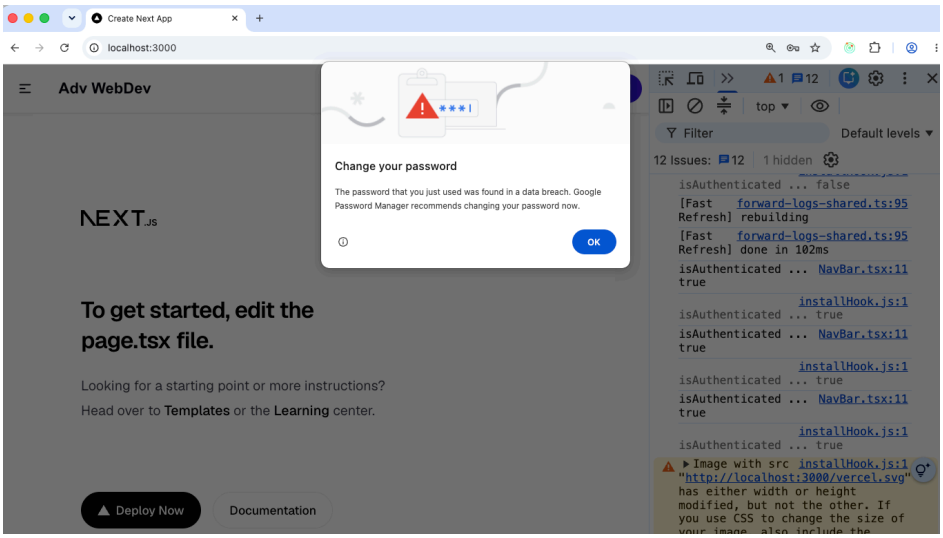




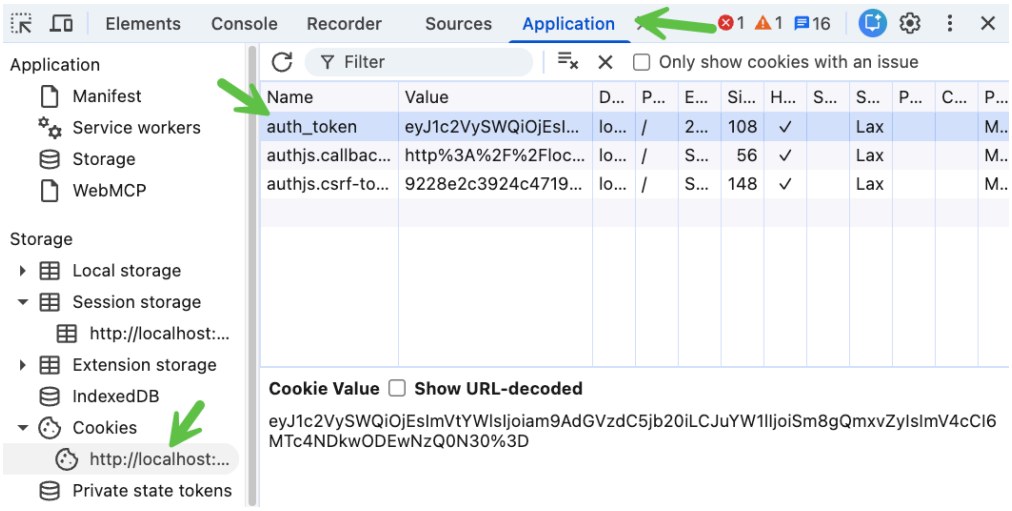
Step 7 – Enter user details



Step 8 – you will be redirected to the home page!

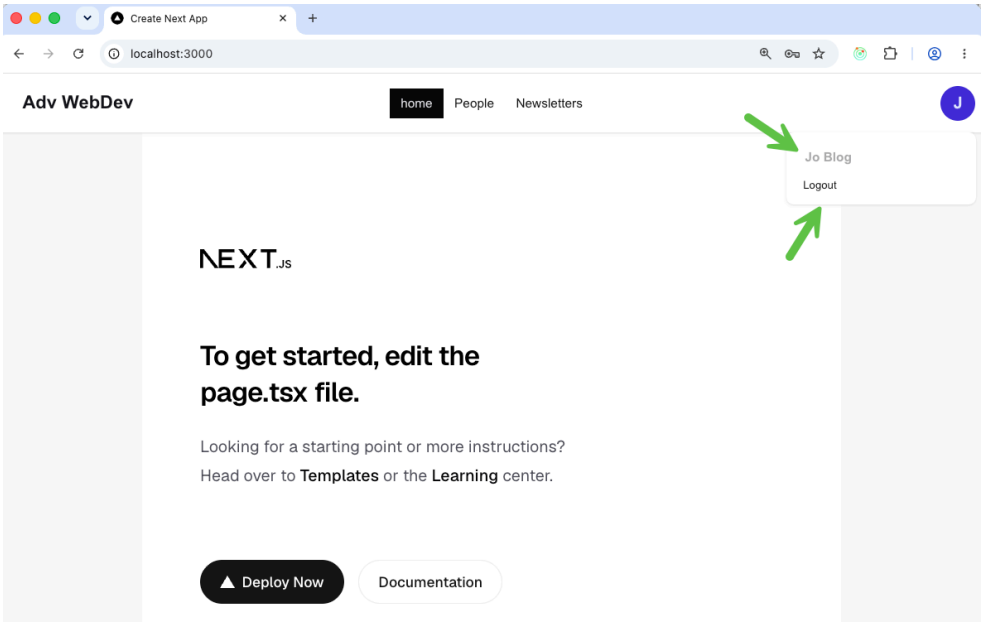


Step 9 – check application tab for the cookie information

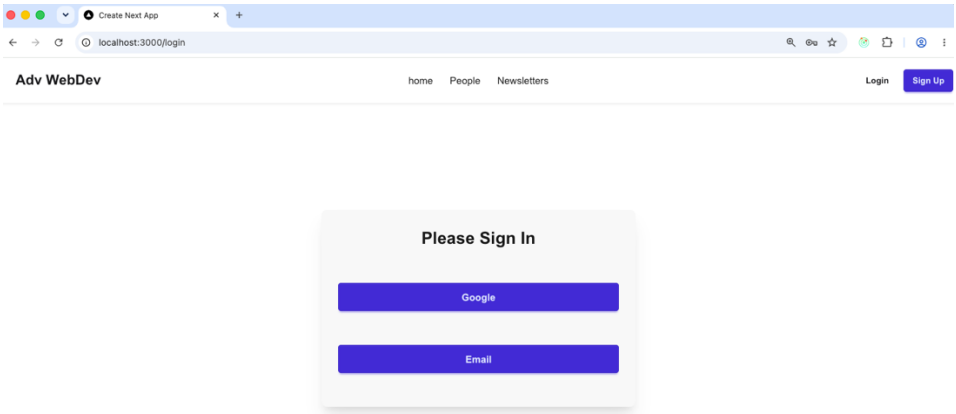


Step 10 - navigate to people and newsletters pages – it should be OK

Step 11– click logout



Step 12 – you will be redirected to login page

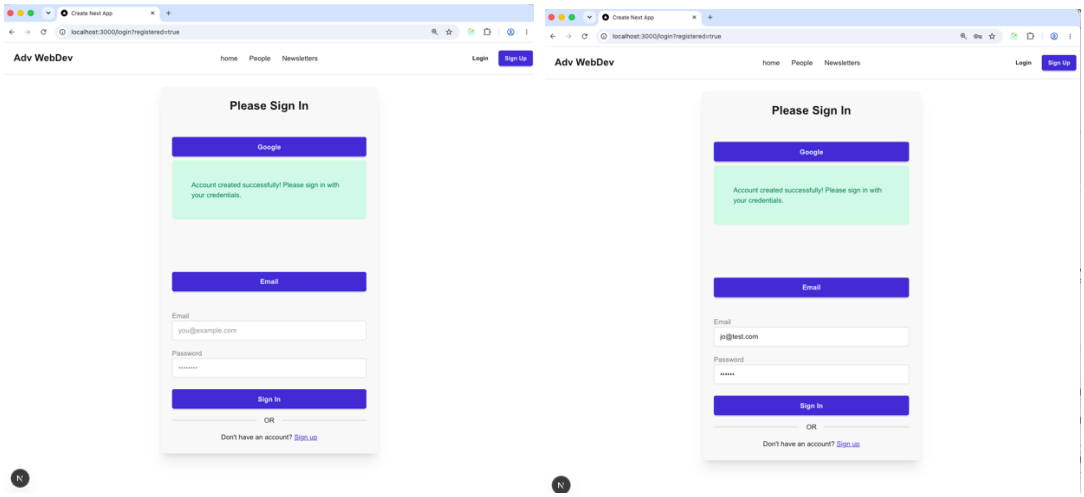


Part 2: Auth.js Credentials implementation

So far, we have implemented custom authenticated system, which is nice, but the more secure way is to use industry standard protocol. Auth.js provides us just that. We have implemented Google Provider supported by Auth.js. we will modify our custom authenticated system to use Auth.js custom credentials next!

Expected output:

After account created:



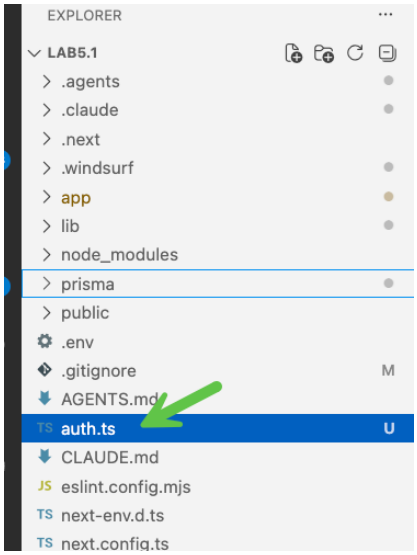
We used Auth.js (NextAuth) for Google OAuth in our last lab exercises, we will use Auth.js to handle the sessions, cookies, and tokens for both Google AND Credentials. We will keep `verifyPassword` from our `lib/auth.js`, but throw away `setAuthCookie`, `generateToken`, and `getCurrentUser`. `Auth.js` handles all of that automatically.

As we will make a lot of changes, I will start with a new folder called lab5.1 and clone lab5 into this folder.

34. Open the `proxy.ts` file, modify as follow:

```ts
TS proxy.ts > ...
1   //export { auth as proxy } from "@/auth";
2
3   import { auth } from "@/auth";
4
5   import { NextResponse } from "next/server";
6
7   export const proxy = auth(async (req) => {
8     const isLoggedIn = !!req.auth; // Auth.js automatically attaches session info to req.auth
9     const currentPath = req.nextUrl.pathname;
10
11    // Paths that are accessible without logging in
12    const isAuthRoute = currentPath === "/login" || currentPath === "/register";
13
14    // 1. Redirect unauthenticated users away from protected pages to /login
15    if (!isLoggedIn && !isAuthRoute) {
16      const newUrl = new URL("/login", req.nextUrl.origin);
17      return NextResponse.redirect(newUrl);
18    }
19
20    // 2. Redirect logged-in users away from /login or /register to homepage
21    if (isLoggedIn && isAuthRoute) {
22      const newUrl = new URL("/", req.nextUrl.origin);
23      return NextResponse.redirect(newUrl);
24    }
25
26    return NextResponse.next();
27  });
28
29  export const config = {
30    matcher: ["/((?!api|_next/static|_next/image|favicon.ico).*)"],
31  };
32
```

35. Now open the `auth.ts` file in the root folder:



36. Currently we have the following code:

```ts
// auth.ts > ...
import NextAuth from "next-auth";
import Google from "next-auth/providers/google";

export const { handlers, signIn, signOut, auth } = NextAuth({
  providers: [Google],
  callbacks: {
    authorized: async ({ auth }) => {
      // Logged in users are authenticated, otherwise redirect to login page
      return !!auth;
    }
  }
});

```

37. Do the following:
    a. Import – Credentials (line3), verifyPassword (line4) and prisma (line 5)

```ts
// auth.ts > ...
import NextAuth from "next-auth";
import Google from "next-auth/providers/google";
import Credentials from "next-auth/providers/credentials";
import { verifyPassword } from "@/app/lib/auth"; // Reuse bcrypt
import { prisma } from "@/app/lib/prisma";

```

    b. Add Credentials (line 10) inside the providers. This Credentials has a method called 'authorize' (line 16 to line 35):

```ts
providers: [
  Google,
  Credentials({
    name: "Credentials",
    credentials: {
      email: { label: "Email", type: "email" },
      password: { label: "Password", type: "password" },
    },
    async authorize(credentials) {
      if (!credentials?.email || !credentials?.password) return null;

      const user = await prisma.user.findUnique({
        where: { email: credentials.email as string },
      });

      if (!user || !user.password) return null;

      // Use existing verifyPassword helper from lib/auth.js
      const isValid = await verifyPassword(
        credentials.password as string,
        user.password,
      );

      if (!isValid) return null;

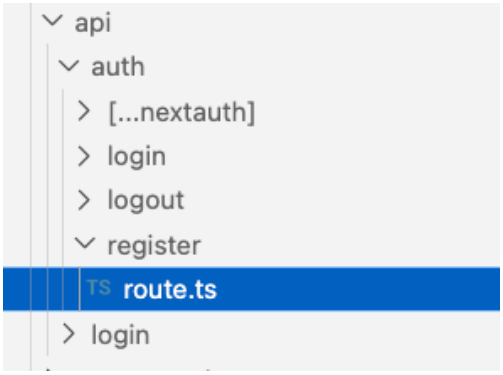
      // Return user - Auth.js creates its OWN secure JWT + cookie!
      return { id: user.id.toString(), email: user.email, name: user.name };
    },
  }),
],
```

c. Set session strategy to JWT (line 38)

```
38    session: { strategy: "jwt" },
39    callbacks: {
40      authorized: async ({ auth }) => {
41        // Logged in users are authenticated, otherwise redirect to login page
42        return !!auth;
43      },
44    },
45  });
46
```

d. Save the file

38. Open the `api/auth/register/route.ts` file



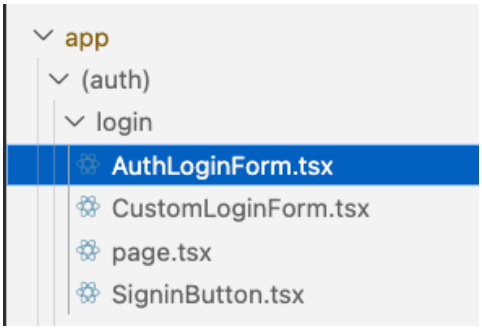
39. Comment out the following:

```
40    //Generate token - no need  for auth
41    // const token = generateToken({
42    //   id: user.id,
43    //   email: user.email,
44    //   name: user.name,
45    // });
```

```
60    // Set httpOnly cookie no need for auth
61    // setAuthCookie(response, token);
```

40. Repeat item 51 above in `api/auth/login/route.ts` file

41. We will create a login form for auth.js. Inside app/(auth)/login folder, create a new file called `AuthLoginForm.tsx`:



42. Open this new file and code the following:
    a. Make it a client component
    b. Import the following

```
3   import { signIn } from "next-auth/react";
4   import { useState } from "react";
5   import { useRouter } from "next/navigation";
6   import Link from "next/link";
7
```

    c. Code the component, and export it (export default)

```
7
8   const AuthLoginForm = () => {
9
10  }
11
```

    d. Inside the component, create 4 states

```
12
13  const [email, setEmail] = useState("");
14  const [password, setPassword] = useState("");
15  const [error, setError] = useState("");
16  const [isSubmitting, setIsSubmitting] = useState(false);
```

    e. Initialise the router

```
13      const router = useRouter();
```

    f. Code a handler function to handle form submission:

```
14
15  const handleSubmit = async (e: React.SubmitEvent<HTMLFormElement>) => {
16    e.preventDefault();
17    setError("");
18    setIsSubmitting(true);
19    // Trigger Auth.js credentials flow directly
20    const res = await signIn("credentials", {
21      email,
22      password,
23      redirect: false, // Prevents automatic redirect so you can handle errors locally
24    });
25
26    if (res?.error) {
27      setError("Invalid email or password");
28    } else {
29      router.push("/"); // Redirect upon success
30      router.refresh();
31    }
32  };
33
```

g. In return block, render:

i. Form

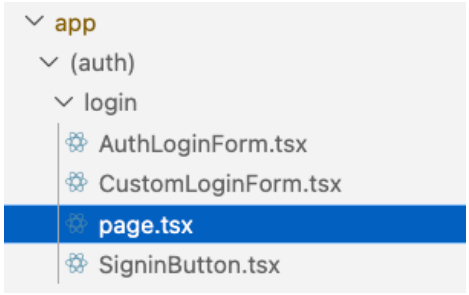
```
<form onSubmit={handleSubmit} className="mt-4">
  {error && (
    <div className="alert alert-error mb-4">
      <span>{error}</span>
    </div>
  )}
  <div className="form-control">
    <label className="label">
      <span className="label-text">Email</span>
    </label>
    <input
      type="email"
      placeholder="you@example.com"
      className="input input-bordered w-full"
      value={email}
      onChange={(e) => setEmail(e.target.value)}
      required
    />
  </div>
  <div className="form-control mt-4">
    <label className="label">
      <span className="label-text">Password</span>
    </label>
    <input
      type="password"
      placeholder="••••••••"
      className="input input-bordered w-full"
      value={password}
      onChange={(e) => setPassword(e.target.value)}
      required
    />
  </div>
  <div className="form-control mt-6">
    <button
      type="submit"
      className="btn btn-primary w-full"
      disabled={isSubmitting}
    >
      {isSubmitting ? (
        <span className="loading loading-spinner loading-sm"></span>
      ) : (
        "Sign In"
      )}
    </button>
  </div>
</form>
<div className="divider">OR</div>
```

ii. Sign up link at the bottom:

```
84          <p className="text-center">
85            Don&apos;t have an account?{" "}
86            <Link href="/register" className="link link-primary">
87              Sign up
88            </Link>
89          </p>
```

iii. Save the file!

h. Open the login page:



i. Here we import this `AuthLoginForm` (line 4)

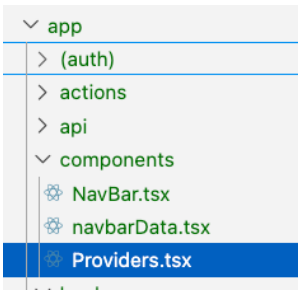
```
2
3   import CustomLoginForm from "./CustomLoginForm";
4   import AuthLoginForm from "./AuthLoginForm";
5   import SigninButton from "./SigninButton";
6   import { useState, useEffect } from "react";
```

j. And replace the Custom form with this one (line 62)

```
60          </div>
61          {/* <div className="mt-4">{elogin ? <CustomLoginForm /> : ""}</div> */}
62          <div className="mt-4">{elogin ? <AuthLoginForm /> : ""}</div>
```

k. Save the file

43. We want our `NavBar` to be able to detect current user. We can use `useSession` hook but it requires the `SessionProvider` context. Open the `Providers.tsx` file



44. Modify the file and add the Session Provider:

app > components > Providers.tsx > ...

```tsx
"use client";

import { AuthProvider } from "../hooks/useAuth";
import { SessionProvider } from "next-auth/react";

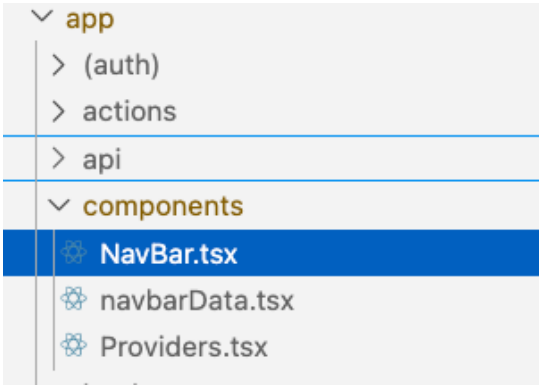
export function Providers({ children }: { children: React.ReactNode }) {
  return (
    <SessionProvider>
      <AuthProvider>{children}</AuthProvider>
    </SessionProvider>
  );
}

```

45. Make sure this Provider is wrapped in your `layout.tsx` file.

```tsx
        <body className="min-h-full flex flex-col">
          <Providers>
            <PeopleProvider>
              <Toaster position="bottom-right" />
              <NavBar />
              {children}
            </PeopleProvider>
          </Providers>
        </body>
```

46. Now we can use the `useSession` hook in our `NavBar` component. Open the `NavBar` :



47. Here we import `useSession` and `signOut`

```tsx
import { useAuth } from "../hooks/useAuth";
import { useSession, signOut } from "next-auth/react";

```

48. Get the session data :

```tsx
  // const { user, logout, isLoading } = useAuth();
  const { data: session } = useSession();

```

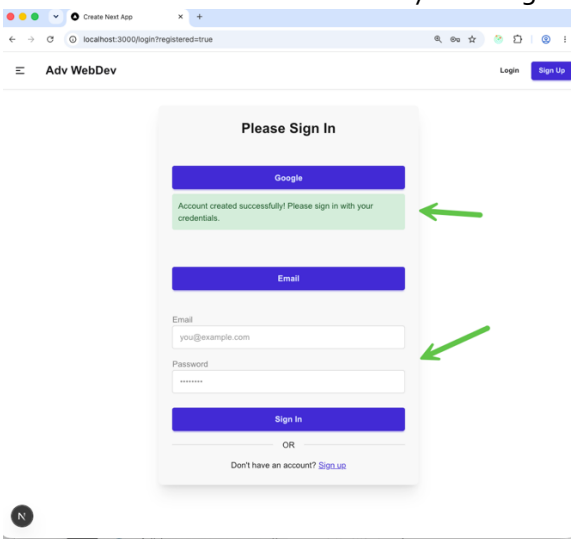
49. Replace the "isAuthenticated" with session user :

```
{session?.user ? (
  <div className="dropdown dropdown-end">
    <div
      tabIndex={0}
      role="button"
      className="btn btn-ghost btn-circle avatar placeholder"
    >
      <div className="bg-primary text-primary-content w-10 rounded-full flex items-center justify-center">
        <span className="text-lg">
          {session?.user?.name?.charAt(0).toUpperCase()}
        </span>
      </div>
    </div>
```

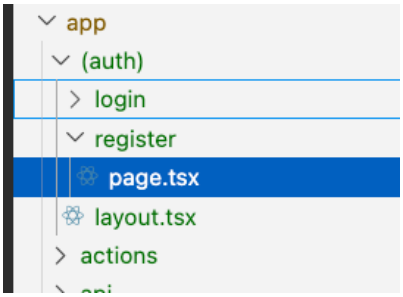
50. Replace the logout function to signOut :

```
<li>
  {/* <button onClick={logout}>Logout</button> */}
  <button onClick={() => signOut({ redirectTo: "/login" })}>
    Logout
  </button>
</li>
```

51. We also need to modify registration form so that we can render the message and open the custom form for the user once they have registered successfully.



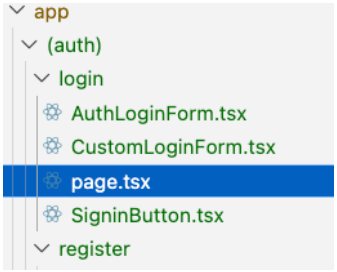
52. First, open the register page:



53. Add "registered=true" in the redirect string:

```
41      try {
42        await register({ email, password, name });
43        router.push("/login?registered=true");
44      } catch (err) {
45        setError(err instanceof Error ? err.message : "Registration failed");
46      } finally {
47        setIsSubmitting(false);
48      }
49    };
50
```

54. Now open the login page



55. Import :

```
8   import { useSearchParams } from "next/navigation";
9
```

56. Initialise it and get the parameter values

```
12
13    const searchParams = useSearchParams();
14    const isJustRegistered = searchParams.get("registered") === "true";
15
```

57. Check this value on mount, and reset the flag so that custom form appears!

```
16    useEffect(() => {
17      if (isJustRegistered) {
18        setElogin(true);
19      }
20    }, []);
21
```

58. Then just code a block to render the message, modify the CSS to your liking.

```
40      {/* Success banner if redirected after registration */}
41      {isJustRegistered && (
42        <div className="p-10 bg-emerald-100 text-emerald-700 mb-15 rounded-md">
43          Account created successfully! Please sign in with your credentials.
44        </div>
45      )}
```

That's it!

Test it to see if it works for you. If for some reasons it does not work for you and there's hidden bugs, go to AI – and prompt for explanations and solutions!

Your next task is to commit your code to CMP Stugit and name your repo as "<your-username>lab5.0"